

TD2 — Parallélisation OpenMP

d'un code de Dynamique Moléculaire

Programmation parallèle — OpenMP

Semestre 4 — 2026

Environnement de développement

OS	macOS (Apple Silicon)
Compilateur	GCC 15 via Homebrew
Profilage	<code>sample</code> (équivalent macOS de <code>perf</code>)
Processeur	4 cœurs Performance + 4 cœurs Efficiency

Table des matières

1	TD2 — Parallélisation OpenMP d'un code de Dynamique Moléculaire	3
1.1	Question 1 : Étude du code et déroulement du programme	3
1.1.1	De quoi parle ce code ?	3
1.1.2	Comment le programme se déroule-t-il ?	3
1.1.3	Les résultats changent-ils entre deux exécutions ?	4
1.1.4	Et les temps d'exécution ?	5
1.2	Question 2 : Profilage et parallélisation de la fonction la plus coûteuse . . .	5
1.2.1	Profilage	5
1.2.2	Analyse de <code>forces()</code> pour la parallélisation	6
1.2.3	Le problème principal : la data race sur <code>f[j]</code>	7
1.2.4	Solution retenue : tableau local par thread	7
1.3	Question 3 : Tests des politiques d'ordonnancement	8
1.3.1	Rappel du fonctionnement des différentes politiques	8
1.3.2	Pourquoi l'équilibrage est crucial pour <code>forces()</code> ?	8
1.3.3	Résultats des tests (NPART=MINI, 1 372 particules)	10
1.3.4	Ce que j'en retiens	11
1.4	Question 4 : Version 1 — Boucle extérieure parallélisée avec <code>single</code>	11
1.4.1	L'idée	11
1.4.2	Structure du code	12
1.4.3	Avantages et limites	12
1.5	Question 5 : Version 2 — Toutes les fonctions parallélisées	12
1.5.1	Le principe	13
1.5.2	Problème rencontré avec GCC : " <code>reduction variable is private</code> <code>in outer context</code> "	13
1.5.3	Solution trouvée	14

1.6	Question 6 : Suppression du goulot d'étranglement (<code>critical</code> → tableau indexé par thread)	15
1.6.1	Le problème avec la version Q2	15
1.6.2	La solution : un tableau 2D indexé par le numéro de thread	15
1.6.3	Comparaison entre les deux approches	16
1.7	Question 7 : Étude d'extensibilité (<code>NPART=MAXI</code>)	16
1.7.1	Résultats	17
1.7.2	Analyse des résultats	17
1.7.3	Conclusion	18

1 TD2 — Parallélisation OpenMP d'un code de Dynamique Moléculaire

Ce TD a été réalisé sur macOS. Quelques adaptations ont été nécessaires par rapport à l'énoncé prévu pour Linux, elles sont expliquées au fil du rapport.

1.1 Question 1 : Étude du code et déroulement du programme

1.1.1 De quoi parle ce code ?

Le programme simule le comportement d'un ensemble de particules qui interagissent entre elles selon un potentiel de Lennard-Jones. C'est un classique de la dynamique moléculaire : on place des atomes dans une boîte, on leur donne des vitesses initiales, puis on laisse la physique faire son travail pendant un certain nombre de pas de temps.

La taille du système est contrôlée à la compilation par la macro `NPART` :

Mode	mm	Nombre de particules ($4 \times \text{mm}^3$)
MINI	7	1 372
MEDIUM	10	4 000
MAXI	15	13 500

1.1.2 Comment le programme se déroule-t-il ?

En regardant le `main.c`, on comprend assez vite la structure. Le programme passe par trois grandes phases :

1. L'initialisation Tout d'abord, on définit les paramètres physiques de la simulation (densité, température de référence, pas de temps `h`, rayon de coupure `rcoff`, etc.).

Ensuite, la fonction `fcc()` place les particules sur un réseau cristallin cubique à faces centrées (FCC). C'est une configuration initiale classique en simulation moléculaire — les

atomes sont bien rangés dans la boîte.

Puis, `mxwell()` attribue des vitesses aléatoires aux particules selon une distribution de Maxwell-Boltzmann, ce qui correspond à un état thermique à la température voulue. Enfin, `dfill()` met toutes les forces initiales à zéro (ce qui est logique, puisqu'on est sur un réseau parfait, les forces s'annulent par symétrie).

2. La boucle principale (20 pas de temps) C'est le cœur de la simulation. À chaque pas de temps, on fait dans l'ordre :

- `domove()` : on déplace les particules en utilisant un schéma d'intégration de type Verlet. On applique aussi des conditions aux limites périodiques (si une particule sort par un côté de la boîte, elle rentre par l'autre).
- `forces()` : c'est la fonction la plus importante. Elle calcule les forces d'interaction entre toutes les paires de particules, et accumule au passage l'énergie potentielle (`epot`) et le viriel (`vir`).
- `mkekin()` : met à l'échelle les forces par le pas de temps, met à jour les vitesses, et calcule l'énergie cinétique.
- `velavg()` : calcule la vitesse moyenne des particules.
- `Thermostat` : pendant les 10 premiers pas, on rééchelonne les vitesses avec `dscal()` pour maintenir la température autour de la valeur cible (sinon, la température dérive).
- `prnout()` : affiche un résumé (énergie cinétique, potentielle, totale, température, pression, vitesse) tous les 5 pas.

3. Mesure du temps Le temps est mesuré avec `omp_get_wtime()` autour de la boucle principale.

1.1.3 Les résultats changent-ils entre deux exécutions ?

Non, les résultats sont parfaitement identiques d'une exécution à l'autre. J'ai lancé le programme deux fois en mode MINI et j'obtiens exactement les mêmes valeurs :

```
1  === Execution 1 (MINI) ===
2      5      1273.0249   -7838.6500   -6565.6250      0.6190    -3.4754
           0.1819   39.9
3      10      1484.7930   -7168.6623   -5683.8693      0.7220    -0.9322
           0.1337   13.0
4      15      1142.9401   -6801.2750   -5658.3349      0.5558      0.4083
           0.1711   32.5
```

```

5      20      1072.1641   -6732.9109   -5660.7468      0.5214      0.6807
        0.1651   29.9
6
7  === Execution 2 (MINI) ===
8      5      1273.0249   -7838.6500   -6565.6250      0.6190     -3.4754
        0.1819   39.9
9      ...   (identique)

```

Deux exécutions identiques (MINI)

C'est normal : le générateur de nombres pseudo-aléatoires dans `random.c` utilise une graine fixe (PMOD/ADDEND). Du coup, la séquence de nombres aléatoires est toujours la même, l'initialisation des vitesses est toujours la même, et comme la simulation est déterministe, tout le déroulement est reproductible.

Par contre, les résultats changent bien sûr quand on passe d'une configuration à une autre (MINI vs MEDIUM vs MAXI), puisque le nombre de particules est différent.

1.1.4 Et les temps d'exécution ?

J'observe les temps suivants pour un seul thread :

Mode	Particules	Temps
MINI	1 372	~0.19 s
MEDIUM	4 000	~1.19 s
MAXI	13 500	~12.80 s

On voit que le temps augmente très vite avec le nombre de particules. C'est cohérent : la fonction `forces()` parcourt toutes les paires (i, j) , donc sa complexité est en $\mathcal{O}(n^2)$, et c'est elle qui domine le temps de calcul (on va le voir en Q2).

1.2 Question 2 : Profilage et parallélisation de la fonction la plus coûteuse

1.2.1 Profilage

L'énoncé demande d'utiliser `perf record` et `perf report`. Cependant, **perf est un outil spécifique à Linux** qui repose sur les compteurs de performance du noyau Linux (`perf_events`). Il n'existe tout simplement pas sur macOS.

```

1 # SUR LINUX (ce qui est demande dans l'enonce) :
2 perf record -e cpu-clock ./md      # enregistre les echantillons
3 perf report                        # affiche le rapport interactif
4
5 # SUR macOS (ce que j'ai utilise a la place) :
6 ./md &                             # lancer en arriere-plan
7 sample $! 8 -f rapport.txt        # echantillonner pendant 8
    secondes

```

Profilage : Linux vs macOS

La commande `gcc` sur Mac est en réalité Apple Clang, qui ne supporte pas OpenMP. Il faut installer le vrai GCC via Homebrew (`brew install gcc`), ce qui donne la commande `gcc-15`.

Voici ce que donne le profilage (avec `sample` sur macOS, mais `perf` donnerait le même type de résultat) :

```

1 Call graph:
2      6149 echantillons au total
3      6141 forces      -> 99.87%
4          5 mkekin      -> 0.08%
5          2 domove      -> 0.03%
6          1 velavg      -> 0.02%

```

Le résultat est sans appel : **forces()** représente 99.87% du temps de calcul. C'est cette fonction qu'il faut paralléliser en priorité. Les autres fonctions sont négligeables en comparaison.

C'est logique quand on y pense : `forces()` contient une double boucle sur toutes les paires de particules (complexité $\mathcal{O}(n^2)$), alors que les autres fonctions ne font qu'une simple boucle sur les particules ($\mathcal{O}(n)$).

1.2.2 Analyse de `forces()` pour la parallélisation

Regardons le code de plus près. La boucle extérieure parcourt les particules `i`, et pour chaque `i`, la boucle intérieure parcourt les particules `j > i` pour calculer les interactions

par paires.

L'idée naturelle est de paralléliser la boucle extérieure en `i` avec `#pragma omp for`. Mais avant ça, il faut bien identifier le statut de chaque variable pour éviter les bugs de concurrence :

Variable	Type d'accès	Statut OpenMP	Pourquoi ?
<code>x[]</code>	lecture seule	<code>shared</code>	Positions non modifiées dans <code>fo</code>
<code>f[]</code>	lecture/écriture	<code>shared</code> mais problématique !	Voir ci-dessous
<code>epot</code>	accumulation (<code>+=</code>)	<code>reduction(+:epot)</code>	Somme partielle par thread
<code>vir</code>	accumulation (<code>--</code>)	<code>reduction(+:vir)</code>	Idem
<code>fxi, fyi, fzi</code>	privées	<code>automatic private</code>	Déclarées dans la boucle
<code>i</code>	indice de boucle	<code>private</code>	Géré par <code>omp for</code>
<code>j</code>	indice interne	<code>private</code>	Déclaré dans le corps
<code>xx, yy, zz...</code>	temporaires	<code>private</code>	Déclarées dans le bloc

1.2.3 Le problème principal : la data race sur `f[j]`

C'est le point le plus délicat. Dans la boucle interne, on a ces lignes :

```

1 f[j]    -= xx * r148;
2 f[j+1] -= yy * r148;
3 f[j+2] -= zz * r148;
```

Imaginons que le thread 0 traite la particule `i=0` et interagit avec `j=42`, pendant que le thread 1 traite la particule `i=6` et interagit aussi avec `j=42`. Les deux threads vont essayer de modifier `f[42]` en même temps : c'est une **data race**, et le résultat sera faux.

Pour `f[i]`, il n'y a pas de problème car chaque thread a ses propres valeurs de `i` (réparties par `omp for`). Mais `f[j]` peut être touché par n'importe quel thread.

1.2.4 Solution retenue : tableau local par thread

Chaque thread travaille sur son propre tableau `f_local[]` initialisé à zéro. Pendant le calcul, au lieu d'écrire dans `f[j]`, on écrit dans `f_local[j]`. Il n'y a donc plus aucun conflit. À la fin, on additionne les tableaux locaux dans le tableau partagé `f[]` via une section `#pragma omp critical`.

J'ai vérifié que les résultats restent identiques après parallélisation :


```
1 1 thread -> 1072.1641 -6732.9109 -5660.7468 <--  
    identique au seq.  
2 4 threads -> 1072.1641 -6732.9109 -5660.7468 <--  
    identique
```

1.3 Question 3 : Tests des politiques d'ordonnement

Pour cette question, on utilise `schedule(runtime)` dans le code et on contrôle la politique via la variable d'environnement `OMP_SCHEDULE`. Ça permet de tester différentes politiques sans recompiler.

1.3.1 Rappel du fonctionnement des différentes politiques

- **static** : les itérations sont découpées en blocs de taille N/P (N itérations, P threads) et distribuées une fois pour toutes au démarrage, en round-robin. C'est le plus simple et celui avec le moins d'overhead, mais il ne s'adapte pas si la charge n'est pas uniforme.
- **static,k** : comme **static**, mais avec des blocs de taille k au lieu de N/P . Avec $k=1$, chaque thread reçoit les itérations en entrelacement (0 pour thread 0, 1 pour thread 1, 2 pour thread 0, etc.). Ça aide à mieux répartir la charge si elle est déséquilibrée.
- **dynamic** : les itérations sont distribuées à la volée — quand un thread finit son bloc, il en prend un nouveau dans une file d'attente. Très bon pour l'équilibrage de charge, mais l'overhead de la file d'attente peut coûter cher si les blocs sont petits.
- **dynamic,k** : comme **dynamic** avec des blocs de taille k . Plus k est grand, moins il y a d'overhead, mais l'équilibrage est moins fin.
- **guided** : similaire à **dynamic**, mais la taille des blocs décroît au fil du temps (commence avec de gros blocs, finit avec des blocs de taille k). L'idée est de réduire l'overhead au début (gros blocs) tout en gardant un bon équilibrage à la fin (petits blocs).

1.3.2 Pourquoi l'équilibrage est crucial pour `forces()` ?

C'est un point important à comprendre. La boucle interne de `forces()` fait :

```
1  for (j = i+3; j < n3; j += 3) { ... }
```

Quand *i* est petit (par ex. *i*=0), la boucle interne parcourt presque toutes les particules. Quand *i* est grand (vers la fin), la boucle interne ne parcourt que quelques particules. On a donc une **charge triangulaire** : les premières itérations sont beaucoup plus lourdes que les dernières.

Avec un **static** simple, le thread 0 hérite des premières itérations (les plus lourdes) et les derniers threads des itérations légères. Le thread 0 travaille donc beaucoup plus longtemps que les autres : mauvais équilibrage.

1.3.3 Résultats des tests (NPART=MINI, 1 372 particules)

Politique	Threads	Temps (s)
static	1	0.188
static	2	0.135
static	4	0.094
static	8	0.068
static,1	1	0.171
static,1	2	0.092
static,1	4	0.068
static,1	8	0.063
static,10	1	0.167
static,10	2	0.102
static,10	4	0.067
static,10	8	0.125
dynamic	1	0.161
dynamic	2	0.086
dynamic	4	0.070
dynamic	8	0.051
dynamic,1	1	0.164
dynamic,1	2	0.157
dynamic,1	4	0.048
dynamic,1	8	0.034 ← <i>meilleur !</i>
dynamic,10	1	0.152
dynamic,10	2	0.080
dynamic,10	4	0.043
dynamic,10	8	0.059
guided	1	0.164
guided	2	0.124
guided	4	0.084
guided	8	0.060
guided,1	1	0.159
guided,1	2	0.124
guided,1	4	0.088
guided,1	8	0.072

1.3.4 Ce que j'en retiens

static classique est le moins bon, comme on pouvait s'y attendre avec la charge triangulaire. Le premier thread se retrouve avec les itérations les plus lourdes et fait attendre tout le monde.

static,1 (entrelacé) est nettement meilleur. Grâce à l'entrelacement, les itérations lourdes et légères sont mélangées entre les threads. Par exemple, avec 4 threads : le thread 0 a les itérations 0, 4, 8, 12... (un mélange de lourd et léger), au lieu d'avoir seulement 0, 1, 2, 3... (les plus lourdes).

dynamic,1 donne le meilleur résultat global (**0.034 s** avec 8 threads). C'est logique : chaque thread prend une itération dès qu'il est libre, ce qui garantit un équilibrage quasi parfait. L'overhead de la file d'attente est compensé par le gain d'équilibrage.

guided donne des résultats intermédiaires. Les blocs décroissants aident un peu, mais c'est moins bien adapté que **dynamic,1** pour notre charge triangulaire.

En termes de speedup avec **dynamic,1** : on passe de 0.164 s (1 thread) à 0.034 s (8 threads), soit un **speedup d'environ 4.8×**. C'est correct pour un Mac avec ses cœurs Performance + Efficiency (les cœurs E sont plus lents, ce qui limite un peu).

1.4 Question 4 : Version 1 — Boucle extérieure parallélisée avec **single**

1.4.1 L'idée

Jusqu'ici (Q2), la région parallèle **#pragma omp parallel** était **dans forces()**. Ça veut dire qu'à chaque pas de temps, les threads sont créés, font le calcul des forces, puis sont détruits. Et au pas de temps suivant, on recommence... On les crée/détruit 20 fois, ce qui génère un overhead inutile.

L'idée de la Q4 est de mettre la région parallèle **autour de la boucle temporelle** dans **main.c**. Les threads sont créés une seule fois au début et restent actifs pendant toute la simulation.

Dans cette version 1, seule **forces()** est réellement parallélisée (elle contient un **#pragma omp for** à l'intérieur). Les autres fonctions (**domove**, **mkekin**, **velavg**, etc.) sont exécutées par un seul thread grâce à **#pragma omp single** — les autres threads attendent à la barrière implicite.

1.4.2 Structure du code

```

1  #pragma omp parallel default(shared) private(move)
2  {
3      for (move = 1; move <= movemx; move++) {
4
5          #pragma omp single
6          domove(3*npart, x, vh, f, side);    // un seul thread
7
8          forces(npart, x, f, side, rcoeff);  // tous les threads (
           omp for interne)
9
10         #pragma omp single
11         {
12             ekin = mkekin(...);              // un seul thread
13             vel  = velavg(...);              // un seul thread
14             if (...) { dscal(...); ... }      // un seul thread
15             if (...) prnout(...);             // un seul thread
16         }
17     }
18 }

```

Version Q4 : `single` autour des fonctions non parallélisées

1.4.3 Avantages et limites

Suppression de l'**overhead de création/destruction des threads** à chaque pas de temps.

La limite, c'est que pendant l'exécution de `domove`, `mkekin`, `velavg`, etc., tous les threads sauf un sont **inactifs**. Certes, ces fonctions ne représentent que 0.13% du temps total, mais quand on augmente le nombre de threads, cette partie séquentielle commence à peser (c'est la **loi d'Amdahl**). C'est ce qu'on améliore en Q5.

1.5 Question 5 : Version 2 — Toutes les fonctions parallélisées

1.5.1 Le principe

On reprend la structure de Q4 (une seule région parallèle autour de la boucle), mais cette fois **toutes les fonctions** du corps de boucle contiennent un `#pragma omp for` en interne. Plus besoin de `single` (sauf pour le `prnout` qui fait du `printf`).

Voici ce qui a été fait dans chaque fonction :

Fonction	Comment c'est parallélisé	Difficultés
<code>domove</code>	<code>#pragma omp for direct</code>	Aucune — itérations indépendantes
<code>forces</code>	<code>#pragma omp for + tableau thread-indexé (Q6)</code>	Data race sur <code>f[j]</code> (voir Q2 et Q6)
<code>mkekin</code>	<code>#pragma omp for + somme partielle</code>	Réduction sur variable locale (voir ci-c)
<code>velavg</code>	<code>#pragma omp for + somme partielle</code>	Idem + variable globale <code>count</code>
<code>dscal</code>	<code>#pragma omp for direct</code>	Chaque <code>sx[i] *= sa</code> est indépendant
<code>prnout</code>	<code>#pragma omp single</code>	On ne parallélise pas du <code>printf</code> !

1.5.2 Problème rencontré avec GCC : "reduction variable is private in outer context"

En essayant de paralléliser `mkekin()`, j'ai d'abord tenté d'utiliser une réduction classique :

```

1  double mkekin(...) {
2      double sum = 0.0;
3      #pragma omp for reduction(+:sum)    // ERREUR !
4      for (i = 0; ...) { sum += ...; }
5      return sum / hsq;
6  }
```

Tentative qui echoue

error: reduction variable 'sum' is private in outer context

Le problème : `sum` est une variable locale de `mkekin()`, et comme `mkekin()` est appelée depuis l'intérieur d'une région parallèle (celle de `main.c`), cette variable locale est **implicitement privée**. Or, OpenMP ne permet pas de faire une réduction sur une variable déjà privée : il ne saurait pas comment combiner les résultats entre les threads.

1.5.3 Solution trouvée

J'utilise une **variable static** (qui est partagée entre les threads car elle est en mémoire globale) et des **sommes partielles privées** :

```

1  double mkekin(...) {
2      double partial_sum = 0.0;           // chaque thread a la
        sienne
3      static double total_sum;           // partagée (static ->
        globale)
4
5      #pragma omp single
6      total_sum = 0.0;                   // un seul thread remet a
        zero
7
8      #pragma omp for schedule(static)
9      for (i = 0; i < 3*npart; i++) {
10         f[i] *= hsq2;
11         vh[i] += f[i];
12         partial_sum += vh[i] * vh[i];
13     }
14
15     #pragma omp atomic                   // une seule fois par
        thread !
16     total_sum += partial_sum;
17
18     #pragma omp barrier                 // on attend tout le monde
19
20     return total_sum / hsq;            // tous retournent la meme
        valeur
21 }
```

Solution : variable static + sommes partielles + atomic

Le `#pragma omp atomic` n'est exécuté qu'une seule fois par thread (pas à chaque itération de la boucle), donc l'overhead est négligeable.

La même technique est utilisée dans `velavg()` pour les variables `vel` et `count`.

1.6 Question 6 : Suppression du goulot d'étranglement (`critical` → tableau indexé par thread)

1.6.1 Le problème avec la version Q2

Dans la première parallélisation de `forces()` (Q2), on utilisait un tableau local `f_local[]` par thread, puis on faisait une accumulation finale avec `#pragma omp critical` :

```

1 // Version Q2 -- chaque thread fait ca a la fin :
2 #pragma omp critical
3 {
4     for (k = 0; k < n3; k++)
5         f[k] += f_local[k];
6 }
```

Version Q2 — goulot d'étranglement

Le souci, c'est que `critical` est un **verrou** : un seul thread à la fois peut exécuter ce bloc. Avec 8 threads, les threads passent les uns après les autres, chacun parcourant le tableau de taille `n3`. Le temps total d'accumulation est donc $\mathcal{O}(n3 \times \text{nthreads})$ en séquentiel : c'est un goulot d'étranglement qui limite la scalabilité.

L'énoncé nous dit d'ailleurs que *"les instructions atomic constituent un goulot d'étranglement sur la plupart des systèmes"*, et `critical` est encore pire que `atomic` puisqu'il bloque un bloc entier.

1.6.2 La solution : un tableau 2D indexé par le numéro de thread

Au lieu que chaque thread alloue son propre `f_local[]`, on alloue un seul grand tableau partagé `f_all` de taille `nthreads × n3`. Chaque thread écrit dans **sa tranche** :

```

1 f_all : [ tranche thread 0 ][ tranche thread 1 ][ ... ][ tranche
          P-1 ]
2         ^                               ^
3         f_all + 0*n3                   f_all + 1*n3
```

Disposition mémoire du tableau `f_all`

Le thread `tid` écrit dans `f_all[tid * n3 + j]` au lieu de `f[j]`. Comme chaque thread écrit dans une zone mémoire différente, il n'y a **aucun conflit**, **aucun besoin de synchronisation** pendant le calcul.

Et surtout, l'accumulation finale est elle-même **parallélisable** :

```

1 #pragma omp for schedule(static)
2 for (i = 0; i < n3; i++) {
3     for (t = 0; t < nthreads; t++)
4         f[i] += f_all[t * n3 + i];
5 }
```

Accumulation parallele (Q6)

Chaque thread accumule une portion du tableau `f[]`. Pas de `critical`, pas de `atomic`. Le temps d'accumulation est maintenant $\mathcal{O}(n3)$ en parallèle, au lieu de $\mathcal{O}(n3 \times nthreads)$ en séquentiel.

1.6.3 Comparaison entre les deux approches

	Q2 (avec <code>critical</code>)	Q6 (tableau par thread)
Pendant le calcul	Pas de conflit ✓	Pas de conflit ✓
Accumulation finale	Séquentielle (<code>critical</code>) ×	Parallèle (<code>omp for</code>) ✓
Mémoire	$\mathcal{O}(n3)$ par thread	$\mathcal{O}(n3 \times nthreads)$ — idem en pratique
Scalabilité	Limitée par la section critique	Bonne

Le tableau `f_all` est déclaré `static` dans `forces()` pour ne pas le réallouer à chaque pas de temps (la fonction est appelée 20 fois). On ne le réalloue que si le nombre de threads change.

1.7 Question 7 : Étude d'extensibilité (NPART=MAXI)

Pour cette étude, j'ai compilé en mode `NPART=MAXI` (13 500 particules) et mesuré le temps d'exécution en faisant varier le nombre de threads de 1 à 8, avec la politique `dynamic,1` qui avait donné les meilleurs résultats en Q3.

1.7.1 Résultats

Threads	Temps (s)	Speedup	Efficacité
1	12.78	1.00×	100.0%
2	6.54	1.95×	97.6%
3	4.50	2.84×	94.6%
4	3.58	3.57×	89.2%
5	3.33	3.84×	76.8%
6	3.07	4.16×	69.4%
7	2.88	4.44×	63.4%
8	2.65	4.83×	60.4%

Le speedup : $S_P = T_1/T_P$ et l'efficacité : $E_P = S_P/P$.

1.7.2 Analyse des résultats

Jusqu'à 4 threads, le speedup est très bon : on atteint 3.57× pour 4 threads, soit une efficacité de 89%. C'est proche du linéaire, ce qui montre que `forces()` se parallélise bien et que l'overhead de synchronisation reste faible.

Au-delà de 4 threads, on observe un net décrochage. On passe de 89% d'efficacité (4 threads) à seulement 60% (8 threads). Plusieurs raisons expliquent ça :

1. **La loi d'Amdahl.** Même si `forces()` représente 99.87% du temps, il reste 0.13% de code séquentiel (les barrières, les `single`, les `atomic`). En théorie, le speedup maximal avec 99.87% parallèle serait :

$$S_{\max} = \frac{1}{0.0013 + \frac{0.9987}{P}} \approx 7.9 \quad \text{pour } P = 8$$

On est loin en dessous de ce maximum théorique (4.83 vs 7.9), ce qui veut dire que d'autres facteurs jouent aussi.

2. **L'architecture Apple Silicon.** Mon Mac a un processeur avec 4 cœurs « Performance » (rapides) et 4 cœurs « Efficiency » (plus lents, environ 2× moins rapides). Avec 4 threads, on utilise les 4 cœurs P et tout va bien. Avec 8 threads, les 4 threads supplémentaires tournent sur les cœurs E qui sont plus lents, et comme il y a des barrières de synchronisation dans le code, les threads rapides doivent **attendre** les threads lents. Ça plombe l'efficacité. Sur une machine Linux avec 8 cœurs homogènes, les résultats seraient sûrement meilleurs.

3. **L’overhead de synchronisation.** Avec plus de threads, les `#pragma omp barrier`, `atomic`, et la distribution des itérations par `dynamic` coûtent un peu plus cher (plus de contention sur les accès mémoire partagée).
4. **La bande passante mémoire.** Le tableau `f_all` fait $8 \text{ threads} \times 40\,500 \text{ doubles} \approx 2.5 \text{ Mo}$. Ça commence à sortir du cache L2, ce qui implique des accès mémoire plus lents.

1.7.3 Conclusion

Le code se parallélise bien pour un nombre modéré de threads (2 à 4), avec un speedup quasi-linéaire. Au-delà, l’architecture hétérogène de mon Mac et les synchronisations limitent les gains. Sur une machine Linux avec 8 cœurs identiques, on obtiendrait probablement un speedup plus proche des $7.9\times$ théoriques.

- La parallélisation de `forces()` est de loin la plus importante (99.87% du temps)
 - c’est le premier truc à faire.
- L’analyse des data races est essentielle : le `f[j]` m’aurait donné des résultats faux si je n’avais pas utilisé un tableau local.
- Les `critical/atomic` sont pratiques mais peuvent devenir des goulots d’étranglement → le tableau indexé par thread (Q6) est une meilleure solution.
- Paralléliser les petites fonctions (Q5) apporte peu en soi, mais améliore la scalabilité en réduisant la fraction séquentielle.